

Scalable Real-Time Inference Platform on AWS

****Formal Project Document****

Field Detail

Author / Maintainer Roshini

Document Version 1.0

Date June 2026

Repository scalable-real-time-inference-platform

Academic Project Abstract

This project focuses on building a scalable, cloud-oriented machine learning inference platform using FastAPI, Docker, Redis, PostgreSQL, and Amazon Web Services design patterns including ECS, Appl...

The application is implemented and load-tested in a containerised local environment that mirrors the AWS service topology. Terraform describes the production target architecture. Documented perform...

Internship Abstract

This internship-focused work centres on developing scalable backend services using FastAPI, Redis, PostgreSQL, Docker, AWS-aligned architecture, and CI/CD workflows. Deliverables include REST APIs,...

The current implementation provides synchronous real-time inference. Asynchronous worker queues are documented as future enhancements.

1. Executive Summary

The Scalable Real-Time Inference Platform is a production-style machine learning inference service exposing a REST API for real-time Iris classification predictions. The system integrates Redis cac...

****Validation status:**** Application behaviour verified locally with reproducible load-test artifacts. AWS resources defined in Terraform and validated via CI; live AWS deployment is optional.

2. Architecture

Target AWS (production design)

Client ? ALB ? ECS Fargate (FastAPI)

??? ElastiCache Redis

??? RDS PostgreSQL

??? S3 (model.joblib)

CloudWatch (logs, metrics, alarms)

GitHub Actions ? ECR ? ECS

Local implementation (validated)

Client ? FastAPI :8000 ? Redis + PostgreSQL
?
models/model.joblib

3. Technology Stack

- Python 3.11+, FastAPI, Pydantic, SQLAlchemy
- scikit-learn, joblib (Iris classifier)
- Redis 7, PostgreSQL 15
- Docker, Docker Compose
- Terraform (AWS), GitHub Actions
- Load testing: hey, Locust

4. API Specification

Endpoint	Method	Description
/health	GET	API, Redis, DB, model status
/predict	POST	Inference with cache_hit, latency_ms, request_id
/metrics	GET	Prometheus-style application metrics
/model-info	GET	Model metadata and input schema

****Input fields:**** sepal_length, sepal_width, petal_length, petal_width (float, 0?30).

5. Database

Table: ****predictions**** ? stores request_id, model_version, input_hash, input_payload, prediction, cache_hit, latency_ms, created_at.

6. Performance Results (Local, Verified)

Metric	Value
Tool	hey (50 concurrent, 30s)
Requests/sec	713.18
p50 latency	65.9 ms
p95 latency	107.7 ms
p99 latency	151.3 ms
Error rate	0%
Cache hit rate	99.94%
Latency reduction (caching)	83%

Evidence: results/metrics_template.md, results/hey_*.txt

7. Deployment Status

Environment	Status
-----	-----
Docker Compose (local)	Deployed and tested
CI/CD (GitHub Actions)	Configured
AWS (live)	IaC ready; optional apply

8. Verification

make up && make verify
make check

Full guide: docs/VERIFICATION.md

9. Maintainer

****Roshini**** ? responsible for repository and formal documentation updates for future institutional enquiries.

10. References

- docs/FORMAL_PROJECT_DOCUMENTATION.md (complete record)
- docs/DOCUMENTATION_INDEX.md
- docs/LETTER_SUPPORT_SUMMARY.md

End of document